Diya Nagdev
D15A - 04
ML-DL Experiment 4

Implement K-Nearest Neighbors (KNN) and evaluate model performance

1. Dataset Source

The dataset used for this experiment is the **Fake Social Media Account Detection**

Dataset, obtained from Kaggle.

Source Link:
https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts?resource=download
This dataset contains Instagram profile features used to classify accounts as fake or genuine.

2. Dataset Description

The **Fake Social Media Account Detection Dataset** consists of various attributes related to social media profiles. The dataset is designed for **binary classification**, where the objective is to identify whether an account is fake.

Dataset Characteristics

- **Features may include:**
    - number of followers
    - number of following
    - number of posts
    - account activity indicators
    - profile characteristics
- **Target Variable**
    - fake

    - **0 → Genuine Account**
    - **1 → Fake Account**

- **Data Type:** Numerical and categorical features
- **Nature of Dataset:** Real-world structured data related to social media profiles.

This dataset helps train machine learning models to detect suspicious or fake accounts on social platforms.

3. Mathematical Formulation of KNN

KNN is a **supervised learning algorithm** used for **classification and regression**.

It works based on **distance between data points**.

Step 1: Choose K

K represents the **number of nearest neighbors** used for classification.

Example values:

- K = 3
- K = 5
- K = 7

Small K → sensitive to noise
Large K → smoother decision boundary

Step 2: Distance Calculation

KNN calculates the **distance between the test sample and training samples**.

The most common metric is **Euclidean Distance**.

$d(x, y) = \sqrt{\Sigma (xi - yi)^2}$, i = 1 to n

Where:

- xix_ixi = feature value of test sample
- yiy_iyi = feature value of training sample
- nnn = number of features

Distance represents **how similar two accounts are**.

Step 3: Find K Nearest Neighbors

The algorithm computes distance between the test account and **all training accounts**, then selects the **K closest ones**.

Step 4: Majority Voting

The class that appears most frequently among the K neighbors is selected as the prediction.

Example:

Neighbor	Class
1	Fake
2	Fake
3	Genuine

Prediction → **Fake**

Why KNN Works for Fake Account Detection

Fake accounts often show patterns such as:

- Very high **following but low followers**
- **Few posts**
- **Empty descriptions**
- **No profile picture**

KNN detects such patterns by comparing them with known fake and genuine accounts.

4. Algorithm Limitations

1. **Slow for large datasets**
   Distance calculation must be done for every sample.

2. **Memory intensive**
   Entire training data must be stored.

3. **Sensitive to feature scaling**
   Features with larger values dominate distance.

4. **Choice of K affects performance**

   Small K → Overfitting
   Large K → Underfitting

5. **Affected by irrelevant features**

5. Methodology / Workflow

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion
from sklearn.preprocessing import StandardScaler
```

Step 2: Load Dataset

```
import pandas as pd

df = pd.read_csv('/content/data/train.csv')
df.head()
```

	profile pic	nums/length username	fullname words	nums/length fullname	name==username	description length	external URL	private	#posts	#followers	#follows	fake
0	1	0.27	0	0.0	0	53	0	0	32	1000	955	0
1	1	0.00	2	0.0	0	44	0	0	286	2740	533	0
2	1	0.10	2	0.0	0	0	0	1	13	159	98	0
3	1	0.00	1	0.0	0	82	0	0	679	414	651	0
4	1	0.00	2	0.0	0	0	0	1	6	151	126	0

Step 3: Dataset Information

```
df.info()
df.describe()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 576 entries, 0 to 575
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   profile pic           576 non-null    int64
 1   nums/length username  576 non-null    float64
 2   fullname words        576 non-null    int64
 3   nums/length fullname  576 non-null    float64
 4   name==username        576 non-null    int64
 5   description length    576 non-null    int64
 6   external URL          576 non-null    int64
 7   private               576 non-null    int64
 8   #posts                576 non-null    int64
 9   #followers            576 non-null    int64
 10  #follows              576 non-null    int64
 11  fake                  576 non-null    int64
dtypes: float64(2), int64(10)
memory usage: 54.1 KB
```

	0
profile pic	0
nums/length username	0
fullname words	0
nums/length fullname	0
name==username	0
description length	0
external URL	0
private	0
#posts	0
#followers	0
#follows	0
fake	0

dtype: int64

Step 4: Prepare Features and Target

```
X = df.drop("fake", axis=1)
y = df["fake"]
```

Step 5: Train Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Step 6: Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 7: Train KNN Model

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

Step 8: Make Predictions

```
y_pred = knn.predict(X_test)
```

6. Performance Analysis

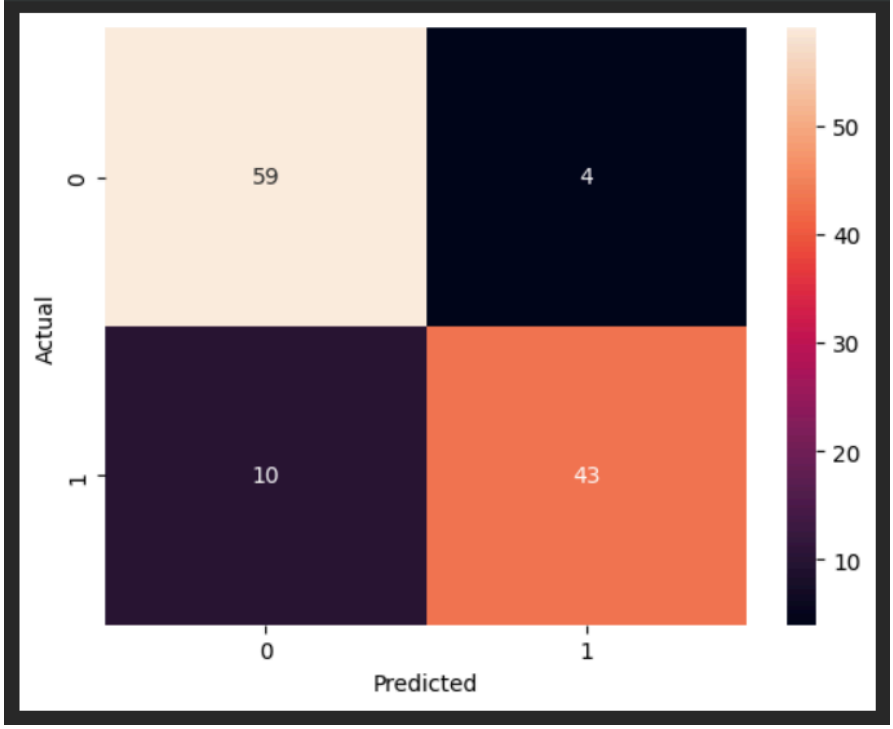
Accuracy

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8793103448275862
```

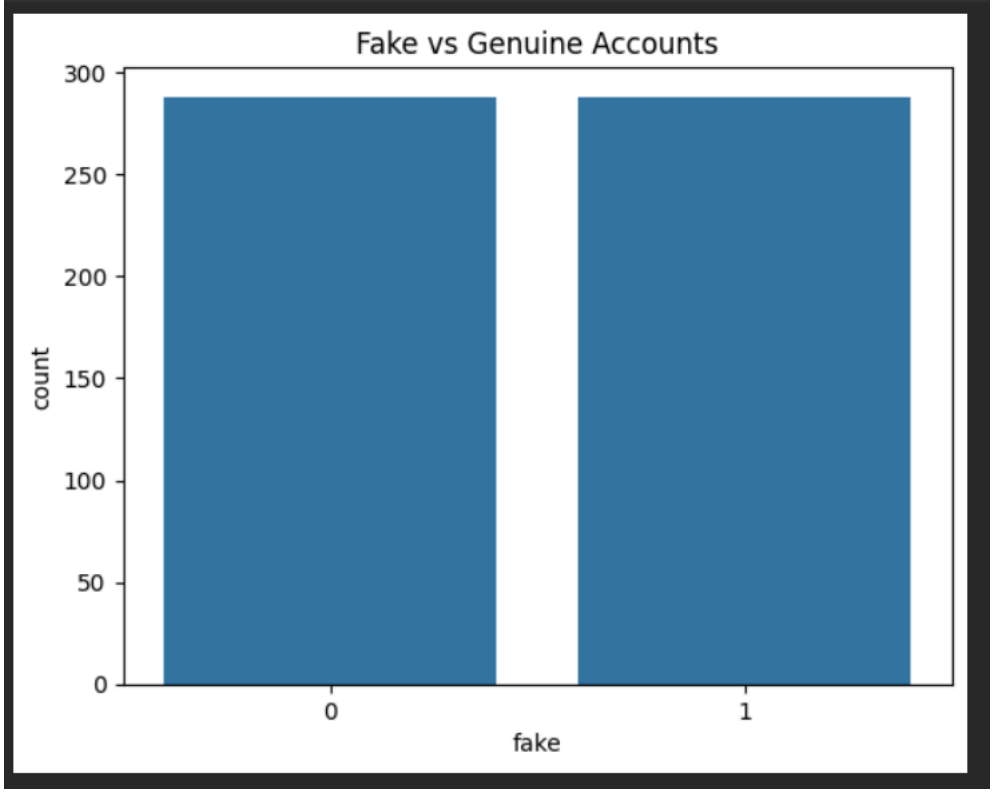
Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Visualization

```
sns.countplot(x=y)
plt.title("Fake vs Genuine Accounts")
plt.show()
```



Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.86	0.94	0.89	63
1	0.91	0.81	0.86	53
accuracy			0.88	116
macro avg	0.88	0.87	0.88	116
weighted avg	0.88	0.88	0.88	116

This provides:

- Precision

- Recall
- F1-score
- Support

7. Conclusion

In this experiment, the **K-Nearest Neighbors (KNN) algorithm** was used to detect fake Instagram accounts using profile features.

The dataset was preprocessed, scaled, and split into training and testing sets. The KNN classifier was trained with **K = 5 neighbors** and evaluated using **accuracy, confusion matrix, and classification report**.

The model successfully classified accounts as **fake or genuine** based on similarities with existing accounts.

This experiment demonstrates how machine learning techniques can be applied to **identify malicious or spam accounts on social media platforms**.

However, KNN may become computationally expensive for very large datasets and requires proper feature scaling for optimal performance.